

- » Recognizing tools and commands
- » Making all your hosts sing with IPConfig and Ping

Chapter 7

TCP/IP Tools and Commands

Most client and server operating systems that support Transmission Control Protocol/Internet Protocol (TCP/IP) come with a suite of commands and tools that are designed to let you examine TCP/IP configuration information and diagnose and correct problems. Although the exact form of these commands varies between Windows and Unix/Linux, most are surprisingly similar. This chapter is a reference to the most commonly used TCP/IP commands.

Using the arp Command

Using the `arp` command allows you to display and modify the Address Resolution Protocol (ARP) cache, which is a simple mapping of IP addresses to MAC addresses. Each time a computer's TCP/IP stack uses ARP to determine the Media Access Control (MAC) address for an IP address, it records the mapping in the ARP cache so that future ARP lookups go faster.

If you use the `arp` command without any parameters, you get a list of the command's parameters. To display the ARP cache entry for a specific IP address, use an `-a` switch followed by the IP address. For example:

```
C:\>arp -a 192.168.168.22
Interface: 192.168.168.21 --- 0x10004
Internet Address Physical Address Type
192.168.168.22 00-60-08-39-e5-a1 dynamic
C:\>
```

You can display the complete ARP cache by using `-a` without specifying an IP address, like this:

```
C:\>arp -a
Interface: 192.168.168.21 --- 0x10004
Internet Address Physical Address Type
192.168.168.9 00-02-e3-16-e4-5d dynamic
192.168.168.10 00-50-04-17-66-90 dynamic
192.168.168.22 00-60-08-39-e5-a1 dynamic
192.168.168.254 00-40-10-18-42-49 dynamic
C:\>
```



TIP

ARP is sometimes useful when diagnosing duplicate IP assignment problems. For example, suppose you can't access a computer that has an IP address of 192.168.1.100. You try to ping the computer, expecting the ping to fail, but lo and behold — the ping succeeds. One possible cause for this may be that two computers on the network have been assigned the address 192.168.1.100, and your ARP cache is pointing to the wrong one. The way to find out is to go to the 192.168.1.100 computer that you want to access, run `ipconfig /all`, and make a note of the physical address. Then return to the computer that's having trouble reaching the 192.168.1.100 computer, run `arp -a`, and compare the physical address with the one you noted. If they're different, two computers are assigned the same IP address. You can then check the Dynamic Host Configuration Protocol (DHCP) or static TCP/IP configuration of the computers involved to find out why.

Using the hostname Command

The `hostname` command is the simplest of all the TCP/IP commands presented in this chapter. It simply displays the computer's host name. For example:

```
C:\>hostname
doug
C:\>
```

Here, the host name for the computer is `doug`. The Windows version of the `hostname` command has no parameters. However, the Unix/Linux versions of `hostname` let you set the computer's host name as well as display it. You do that by specifying the new host name as an argument.

Using the `ipconfig` Command

Using the `ipconfig` command displays information about a computer's TCP/IP configuration. It can also be used to update DHCP and Domain Name System (DNS) settings.

Displaying basic IP configuration

To display the basic IP configuration for a computer, use the `ipconfig` command without any parameters, like this:

```
C:\>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

Connection-specific DNS Suffix . : 
IPv6 Address. . . . . : 2600:6c52:7900:1623:9809:47db:8e66:b0c
Link-local IPv6 Address . . . . . : fe80::9809:47db:8e66:b0c%2
IPv4 Address. . . . . : 192.168.1.15
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : fe80::6238:e0ff:fea2:4f06%2
                           192.168.1.1
```

When you use `ipconfig` without parameters, the command displays the name of the adapter, the domain name used for the adapter, the IP address, the subnet mask, and the default gateway configuration for the adapter. This is the easiest way to determine a computer's IP address. In this example, the IP address is 192.168.1.15.



TIP

If your computer indicates an IP address in the 169.254.x.x block, odds are good that the DHCP server isn't working. 169.254.x.x is the Class B address block that Windows uses when it resorts to IP autoconfiguration. This usually happens only when the DHCP server can't be reached or isn't working.

Displaying detailed configuration information

You can display detailed IP configuration information by using an `/all` switch with the `ipconfig` command, like this:

```
C:\>ipconfig /all

Windows IP Configuration

Host Name . . . . . : WIN1901
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet:

Connection-specific DNS Suffix . :
Description . . . . . : Microsoft Hyper-V Network Adapter
Physical Address. . . . . : 00-15-5D-00-F5-01
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
IPv6 Address. . . . . : 2600:6c52:7900:1623:9809:47db:8e66:b0c(Preferred)
Link-local IPv6 Address . . . . . : fe80::9809:47db:8e66:b0c%2(Preferred)
IPv4 Address. . . . . : 192.168.1.15m (Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : fe80::6238:e0ff:fea2:4f06%2
                             192.168.1.1
DHCPv6 IAID . . . . . : 33559901
DHCPv6 Client DUID. . . . . : 00-01-00-01-26-DD-9F-1D-00-15-5D-00-F5-01
DNS Servers . . . . . : 192.168.1.10
                             192.168.1.11
NetBIOS over Tcpip. . . . . : Enabled
C:\>
```

You can determine a lot of information about the computer from the `ipconfig /all` command. For example:

- » The computer's host name is WIN1901.
- » The computer's IPv4 address is 192.168.1.15, and the subnet mask is 255.255.255.0.
- » The default gateway is a router located at 192.168.1.1.
- » The DNS servers are at 192.168.1.10 and 192.168.1.11.

Renewing an IP lease

If you're having an IP configuration problem, you can often solve it by renewing the computer's IP lease. To do that, use a `/renew` switch, like this:

```
C:\>ipconfig /renew
Windows IP Configuration
Ethernet adapter Local Area Connection:
Connection-specific DNS Suffix . :
IP Address. . . . . : 192.168.1.100
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.1.1
```

When you renew an IP lease, the `ipconfig` command displays the new lease information.



WARNING

This command won't work if you configured the computer to use a static IP address.

Releasing an IP lease

You can release an IP lease by using an `ipconfig` command with the `/release` parameter, like this:

```
C:\>ipconfig /release
Windows IP Configuration
Ethernet adapter Local Area Connection:
Connection-specific DNS Suffix . :
IP Address . . . . . : 0.0.0.0
Subnet Mask. . . . . : 0.0.0.0
Default Gateway. . . . . :
```

As you can see, the DNS suffix and default gateway for the computer are blank, and the IP address and subnet mask are set to 0.0.0.0.

After you release the DHCP lease, you can use an `ipconfig /renew` command to obtain a new DHCP lease for the computer.



WARNING

This command won't work if you configured the computer to use a static IP address.

Flushing the local DNS cache

You probably won't need to do this unless you're having DNS troubles. If you've been tinkering with your network's DNS configuration, though, you may need to flush the cache on your DNS clients so that they'll be forced to reacquire information from the DNS server. You can do that by using a `/flushdns` switch:

```
C:\>ipconfig /flushdns
Windows IP Configuration
Successfully flushed the DNS Resolver Cache.
C:\>
```



Even if you don't need to do this, it's fun just to see the computer read `flushed`. If I worked at Microsoft, you'd be able to revert Windows Vista computers back to XP by using a `/flushVista` switch.

Using the nbtstat Command

`nbtstat` is a Windows-only command that can help solve problems with NetBIOS name resolution. (*nbt* stands for *NetBIOS over TCP/IP*.) You can use any of the switches listed in Table 7-1 to specify what `nbtstat` output you want to display. For example, you can use an `-a` switch to display the cached name table for a specified computer, like this:

```
C:\>nbtstat -a WK07-001
Local Area Connection:
Node IpAddress: [192.168.1.110] Scope Id: []
NetBIOS Remote Machine Name Table
Name Type Status
-----
WK07-001 <00> UNIQUE Registered
WORKGROUP <00> GROUP Registered
WK07-001 <20> UNIQUE Registered
WORKGROUP <1E> GROUP Registered
WORKGROUP <1D> UNIQUE Registered
..._MSBROWSE... <01> GROUP Registered
MAC Address = 00-12-3F-A7-17-BAC:\>
C:\>
```

Table 7-1 lists the switches that you can use with `nbtstat` and explains the function of each switch.

TABLE 7-1**nbtstat Command Switches**

Switch	What It Does
-a name	Lists the specified computer's name table given the computer's name
-A IP-address	Lists the specified computer's name table given the computer's IP address
-c	Lists the contents of the NetBIOS cache
-n	Lists locally registered NetBIOS names
-r	Displays a count of the names resolved by broadcast and via WINS
-R	Purges and reloads the cached name table from the LMHOSTS file
-RR	Releases and then reregisters all names
-S	Displays the sessions table using IP addresses
-s	Displays the sessions table and converts destination IP addresses to computer NetBIOS names

Using the netstat Command

Using the `netstat` command displays a variety of statistics about a computer's active TCP/IP connections. It's a useful tool to use when you're having trouble with TCP/IP applications, such as File Transfer Protocol (FTP), HyperText Transport Protocol (HTTP), and so on.

Displaying connections

If you run `netstat` without specifying any parameters, you get a list of active connections, something like this:

```
C:\>netstat
Active Connections
Proto Local Address Foreign Address State
TCP Doug:1463 192.168.168.10:1053 ESTABLISHED
TCP Doug:1582 192.168.168.9:netbios-ssn ESTABLISHED
TCP Doug:3630 192.168.168.30:9100 SYN_SENT
TCP Doug:3716 192.168.168.10:4678 ESTABLISHED
TCP Doug:3940 192.168.168.10:netbios-ssn ESTABLISHED
C:\>
```

This list shows all the active connections on the computer and indicates the local port used by the connection, as well as the IP address and port number for the remote computer.

You can specify the `-n` switch to display both local and foreign addresses in numeric IP form:

```
C:\>netstat -n
Active Connections
Proto Local Address Foreign Address State
TCP 192.168.168.21:1463 192.168.168.10:1053 ESTABLISHED
TCP 192.168.168.21:1582 192.168.168.9:139 ESTABLISHED
TCP 192.168.168.21:3658 192.168.168.30:9100 SYN_SENT
TCP 192.168.168.21:3716 192.168.168.10:4678 ESTABLISHED
TCP 192.168.168.21:3904 207.46.106.78:1863 ESTABLISHED
TCP 192.168.168.21:3940 192.168.168.10:139 ESTABLISHED
C:\>
```

Finally, you can specify the `-a` switch to display all TCP/IP connections and ports that are being listened to. I won't list the output from that command here because it would run several pages, and I want to do my part for the rainforests. Suffice it to say that it looks a lot like the `netstat` output shown previously, but a lot longer.

Displaying interface statistics

If you use an `-e` switch, `netstat` displays various protocol statistics, like this:

```
C:\>netstat -e
Interface Statistics
Received Sent
Bytes 672932849 417963911
Unicast packets 1981755 1972374
Non-unicast packets 251869 34585
Discards 0 0
Errors 0 0
Unknown protocols 1829
C:\>
```



REMEMBER

The items to pay attention to in this output are the Discards and Errors. These numbers should be zero, or at least close to it. If they're not, the network may be carrying too much traffic or the connection may have a physical problem. If no physical problem exists with the connection, try segmenting the network to see whether the error and discard rates drop.

Using the nslookup Command

The `nslookup` command is a powerful tool for diagnosing DNS problems. You know you're experiencing a DNS problem when you can access a resource by specifying its IP address but not its DNS name. For example, if you can get to `www.ebay.com` by typing `173.223.234.210` in your browser's address bar but not by typing `www.ebay.com`, you have a DNS problem.

Looking up an IP address

The simplest use of `nslookup` is to look up the IP address for a given DNS name. For example, how did I know that `173.223.234.210` was the IP address for `www.ebay.com`? I used `nslookup` to find out:

```
C:\>nslookup ebay.com
Server:      cdns01.comcast.net
Address:     2001:558:feed::1

Non-authoritative answer:
Name:        ebay.com
Addresses:   173.223.234.210
              173.223.234.197
C:\>
```

As you can see, just type **nslookup** followed by the DNS name you want to look up, and `nslookup` issues a DNS query to find out. This DNS query was sent to the server named `cdns-1/cp,cast/met`. It then displayed two IP addresses associated with `ebay.com`: namely, `173.222.234.210` and `173.223.234.197`.



TIP

In some cases, you may find that using an `nslookup` command gives you the wrong IP address for a host name. To know that for sure, of course, you have to know with certainty what the host IP address *should* be. For example, if you know that your server is `203.172.182.10` but `nslookup` returns a completely different IP address for your server when you query the server's host name, something is probably wrong with one of the DNS records.

Using nslookup subcommands

If you use `nslookup` without any arguments, the `nslookup` command enters a subcommand mode. It displays a prompt character (`>`) to let you know that you're in `nslookup` subcommand mode rather than at a normal Windows command prompt. In subcommand mode, you can enter various subcommands to set options or to perform queries. You can type a question mark (`?`) to get a list of these commands. Table 7-2 lists the subcommands you'll use most.

GET ME OUT OF HERE!

One of my pet peeves is that it seems as if every program that uses subcommands chooses a different command to quit the application. I can never remember whether the command to get out of `nslookup` is `quit`, `bye`, or `exit`. I usually end up trying them all. And no matter what program I'm using, I always seem to choose the one that works for some other program first. When I'm in `nslookup`, I use `bye` first. When I'm in `FTP`, I try `exit` first. Arggh! If I were King of the Computer Hill, every program that had subcommands would respond to any of the following commands by exiting the program and returning to a command prompt:

Quit	Sayonara
Exit	Ciao
Bye	Mañana
Leave	Makelikeatree

Of course, the final command to try would be `Andgetouttahere` (in honor of Biff from the *Back to the Future* movies).

TABLE 7-2 **The Most Commonly Used `nslookup` Subcommands**

Subcommand	What It Does
<code>name</code>	Queries the current name server for the specified name.
<code>server name</code>	Sets the current name server to the server you specify.
<code>root</code>	Sets the root server as the current server.
<code>set type=x</code>	Specifies the type of records to be displayed, such as A, CNAME, MX, NS, PTR, or SOA. Specify ANY to display all records.
<code>set debug</code>	Turns on Debug mode, which displays detailed information about each query.
<code>set nodebug</code>	Turns off Debug mode.
<code>set recurse</code>	Enables recursive searches.
<code>set norecurse</code>	Disables recursive searches.
<code>exit</code>	Exits and returns you to a command prompt.

Displaying DNS records

One of the main uses of `nslookup` is to examine your DNS configuration to make sure that it's set up properly. To do that, follow these steps:

1. At a command prompt, type `nslookup` without any parameters.

`nslookup` displays the name of the default name server and displays the `>` prompt.

```
C:\>nslookup
Default Server: ns1.orng.twtelecom.net
Address: 168.215.210.50
>
```

2. Type the subcommand `set type=any`.

`nslookup` silently obeys your command and displays another prompt:

```
> set type=any
>
```

3. Type your domain name.

`nslookup` responds by displaying the name servers for your domain:

```
> lowewriter.com
Server: ns1.orng.twtelecom.net
Address: 168.215.210.50
Non-authoritative answer:
lowewriter.com nameserver = NS000.NS0.com
lowewriter.com nameserver = NS207.PAIR.com
lowewriter.com nameserver = NS000.NS0.com
lowewriter.com nameserver = NS207.PAIR.com
>
```

4. Use a `server` command to switch to one of the domain's name servers.

For example, to switch to the first name server listed in Step 3, type **server `NS000.NS0.com`**. `nslookup` replies with a message that indicates the new default server:

```
> server ns000.ns0.com
Default Server: ns000.ns0.com
Address: 216.92.61.61
>
```

5. Type your domain name again.

This time, nslookup responds by displaying the DNS information for your domain:

```
> lowewriter.com
Server: ns000.ns0.com
Address: 216.92.61.61
lowewriter.com
primary name server = ns207.pair.com
responsible mail addr = root.pair.com
serial = 2001121009
refresh = 3600 (1 hour)
retry = 300 (5 mins)
expire = 604800 (7 days)
default TTL = 3600 (1 hour)
lowewriter.com nameserver = ns000.ns0.com
lowewriter.com nameserver = ns207.pair.com
lowewriter.com MX preference = 50, mail exchanger = sasi.
pair.com
lowewriter.com internet address = 209.68.34.15
>
```

6. Type exit to leave the nslookup program.

You return to a command prompt.

```
> exit
C:\>
```

Wasn't that fun?

Locating the mail server for an email address

If you're having trouble delivering mail to someone, you can use nslookup to determine the IP address of the user's mail server. Then, you can use the ping command to see whether you can contact the user's mail server. If not, you can use the tracert command to find out where the communication breaks down. (See "Using the tracert Command" later in this chapter for more information.)

To find a user's mail server, start nslookup and enter the command **set type=MX**. Then, enter the domain portion of the user's email address. For example, if the user's address is Doug@LoweWriter.com, enter **LoweWriter.com**. nslookup will display the MX (Mail Exchange) information for the domain, like this:

```

C:\>nslookup
Default Server: ns7.attbi.com
Address: 204.127.198.19
> set type=mx
> lowewriter.com
Server: ns7.attbi.com
Address: 204.127.198.19
lowewriter.com MX preference = 50, mail exchanger =
    sasi.pair.com
lowewriter.com nameserver = ns000.ns0.com
lowewriter.com nameserver = ns207.pair.com
ns000.ns0.com internet address = 216.92.61.61
ns207.pair.com internet address = 209.68.2.52
>

```

Here, you can see that the name of the mail server for the `Lowewriter.com` domain is `sasi.pair.com`.

Using the pathping Command

`pathping` is an interesting command that's unique to Windows. It's sort of a cross between the `ping` command and the `tracert` command, combining the features of both into one tool. When you run `pathping`, it first traces the route to the destination address much the way `tracert` does. Then, it launches into a 25-second test of each router along the way, gathering statistics on the rate of data loss to each hop. If the route has a lot of hops, this can take a long time. However, it can help you to spot potentially unreliable hops. If you're having intermittent trouble reaching a particular destination, using `pathping` may help you pinpoint the problem.

The following command output is typical of the `pathping` command. Using an `-n` switch causes the display to use numeric IP numbers only, instead of DNS host names. Although fully qualified host names are convenient, they tend to be very long for network routers, which makes the `pathping` output very difficult to decipher.

```

C:\>pathping -n www.lowewriter.com
Tracing route to lowewriter.com [209.68.34.15]
over a maximum of 30 hops:
0 192.168.168.21
1 66.193.195.81
2 66.193.200.5

```

```

3 168.215.55.173
4 168.215.55.101
5 168.215.55.77
6 66.192.250.38
7 66.192.252.22
8 208.51.224.141
9 206.132.111.118
10 206.132.111.162
11 64.214.174.178
12 192.168.1.191
13 209.68.34.15
Computing statistics for 325 seconds...
Source to Here This Node/Link
Hop RTT Lost/Sent = Pct Lost/Sent = Pct Address
0 192.168.168.21
0/ 100 = 0% |
1 1ms 0/ 100 = 0% 0/ 100 = 0% 66.193.195.81]
0/ 100 = 0% |
2 14ms 0/ 100 = 0% 0/ 100 = 0% 66.193.200.5
0/ 100 = 0% |
3 10ms 0/ 100 = 0% 0/ 100 = 0% 168.215.55.173
0/ 100 = 0% |
4 10ms 0/ 100 = 0% 0/ 100 = 0% 168.215.55.101
0/ 100 = 0% |
5 12ms 0/ 100 = 0% 0/ 100 = 0% 168.215.55.77
0/ 100 = 0% |
6 14ms 0/ 100 = 0% 0/ 100 = 0% 66.192.250.38
0/ 100 = 0% |
7 14ms 0/ 100 = 0% 0/ 100 = 0% 66.192.252.22
0/ 100 = 0% |
8 14ms 0/ 100 = 0% 0/ 100 = 0% 208.51.224.141
0/ 100 = 0% |
9 81ms 0/ 100 = 0% 0/ 100 = 0% 206.132.111.118
0/ 100 = 0% |
10 81ms 0/ 100 = 0% 0/ 100 = 0% 206.132.111.162]
0/ 100 = 0% |
11 84ms 0/ 100 = 0% 0/ 100 = 0% 64.214.174.178]
0/ 100 = 0% |
12 --- 100/ 100 =100% 100/ 100 =100% 192.168.1.191
0/ 100 = 0% |
13 85ms 0/ 100 = 0% 0/ 100 = 0% 209.68.34.15
Trace complete.

```

Using the ping Command

ping is probably the most basic TCP/IP command line tool. Its main purpose is to determine whether you can reach another computer from your computer. It uses Internet Control Message Protocol (ICMP) to send mandatory ECHO_REQUEST datagrams to the specified host computer. When the reply is received back from the host, the ping command displays how long it took to receive the response.

You can specify the host to ping by using an IP address, as in this example:

```
C:\>ping 192.168.168.10
Pinging 192.168.168.10 with 32 bytes of data:
Reply from 192.168.168.10: bytes=32 time<1ms TTL=128
Reply from 192.168.168.10: bytes=32 time<1ms TTL=128
Reply from 192.168.168.10: bytes=32 time<1ms TTL=128
Reply from 192.168.168.10: bytes=32 time<1ms TTL=128
Ping statistics for 192.168.168.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
C:\>
```

By default, the ping command sends four packets to the specified host. It displays the result of each packet sent. Then it displays summary statistics: how many packets were sent, how many replies were received, the error loss rate, and the approximate round-trip time.

You can also ping by using a DNS name, as in this example:

```
C:\>ping www.lowewriter.com
Pinging lowewriter.com [209.68.34.15] with 32 bytes of data:
Reply from 209.68.34.15: bytes=32 time=84ms TTL=53
Reply from 209.68.34.15: bytes=32 time=84ms TTL=53
Reply from 209.68.34.15: bytes=32 time=84ms TTL=53
Reply from 209.68.34.15: bytes=32 time=84ms TTL=53
Ping statistics for 209.68.34.15:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 84ms, Maximum = 84ms, Average = 84ms
C:\>
```

The ping command uses a DNS query to determine the IP address for the specified host, and then pings the host based on its IP address.

The `ping` command has a number of other switches that you'll use rarely, if ever. Some of these switches are available only for some operating systems. To find out which switches are available for your version of Ping, type **ping /?** (Windows) or **man ping** (Unix/Linux).



You can find a very interesting story about the creation of the `ping` command written by the command's author, Mike Muus, at his website at <https://ftp.arl.army.mil/~mike/ping.html>. (Sadly, Mr. Muus was killed in an automobile accident in November 2000.)

Using the `route` Command

Using the `route` command displays or modifies the computer's routing table. For a typical computer that has a single network interface and is connected to a local area network (LAN) that has a router, the routing table is pretty simple and isn't often the source of network problems. Still, if you're having trouble accessing other computers or other networks, you can use the `route` command to make sure that a bad entry in the computer's routing table isn't the culprit.

For a computer with more than one interface and that's configured to work as a router, the routing table is often a major source of trouble. Setting up the routing table properly is a key part of configuring a router to work.

Displaying the routing table

To display the routing table (both IPv4 and IPv6) in Windows, use the `route print` command. In Unix/Linux, you can just use `route` without any command line switches. The output displayed by the Windows and Unix/Linux commands are similar. Here's an example from a typical Windows client computer:

```
C:\>route print
=====
Interface List
8...00 12 3f a7 17 ba..... Intel(R) PRO/100 VE Network Connection
1..... Software Loopback Interface 1
9.....02 00 54 55 4e 01..... Teredo Tunneling Pseudo-Interface
10 ...00 00 00 00 00 00 00 e0 isatap.{D0F85930-01E2-402F-B0FC-31DFF887F06F}
=====
IPv4 Route Table
=====
```



```

Active Routes:
Network Destination Netmask Gateway Interface Metric
0.0.0.0 0.0.0.0 192.168.1.1 192.168.1.110 276
127.0.0.0 255.0.0.0 On-link 127.0.0.1 306
127.0.0.1 255.255.255.255 On-link 127.0.0.1 306
127.255.255.255 255.255.255.255 On-link 127.0.0.1 306
192.168.1.0 255.255.255.0 On-link 192.168.1.110 276
192.168.1.110 255.255.255.255 On-link 192.168.1.110 276
192.168.1.255 255.255.255.255 On-link 192.168.1.110 276
224.0.0.0 240.0.0.0 On-link 127.0.0.1 306
224.0.0.0 240.0.0.0 On-link 192.168.1.110 276
255.255.255.255 255.255.255.255 On-link 127.0.0.1 306
255.255.255.255 255.255.255.255 On-link 192.168.1.110 276

```

```

=====
Persistent Routes:
Network Address Netmask Gateway Address Metric
0.0.0.0 0.0.0.0 192.168.1.1 Default

```

IPv6 Route Table

```

=====
Active Routes:
If Metric Network Destination Gateway
9 18 ::/0 On-link
1 306 ::1/128 On-link
9 18 2001::/32 On-link
9 266 2001:0:4136:e38c:2c6c:670:3f57:fe91/128
On-link
8 276 fe80::/64 On-link
9 266 fe80::/64 On-link
10 281 fe80::5efe:192.168.1.110/128
On-link
8 276 fe80::cca:9067:9427:a911/128
On-link
9 266 fe80::2c6c:670:3f57:fe91/128
On-link
1 306 ff00::/8 On-link
9 266 ff00::/8 On-link
8 276 ff00::/8 On-link

```

```

=====
Persistent Routes:
None
C:\>

```

For each entry in the routing table, five items of information are listed:

»» **The destination IP address**

Actually, this is the address of the destination subnet, and must be interpreted in the context of the subnet mask.

»» **The subnet mask that must be applied to the destination address to determine the destination subnet**

»» **The IP address of the gateway to which traffic intended for the destination subnet will be sent**

»» **The IP address of the interface through which the traffic will be sent to the destination subnet**

»» **The *metric*, which indicates the number of hops required to reach destinations via the gateway**

Each packet that's processed by the computer is evaluated against the rules in the routing table. If the packet's destination address matches the destination subnet for the rule, the packet is sent to the specified gateway via the specified network interface. If not, the next rule is applied.

The computer on which I ran the `route` command in this example is on a private 192.168.1.0 subnet. The computer's IP address is 192.168.1.100, and the default gateway is a router at 192.168.1.1.

Here's how the rules shown in this example are used. Notice that you have to read the entries from the bottom up:

»» **The first rule is for packets sent to 255.255.255.255, with subnet mask 255.255.255.255.** This special IP address is for broadcast packets. The rule specifies that these broadcast packets should be delivered to the local network interface (192.168.1.100).

»» **The next rule is for packets sent to 192.168.1.255, again with subnet mask 255.255.255.255.** These are also broadcast packets and are sent to the local network interface.

»» **The next rule is for packets sent to 192.168.1.100, again with subnet mask 255.255.255.255.** This is for packets that the computer is sending to itself via its own IP address. This rule specifies that these packets will be sent to the local loopback interface on 127.0.0.1.

»» **The next rule is for packets sent to 192.168.1.0, with subnet mask 255.255.255.0.** These are packets intended for the local subnet. They're sent to the subnet via the local interface at 192.169.1.100.

- » **The next rule is for packets sent to the loopback address (127.0.0.1, subnet mask 255.0.0.0).** These packets are sent straight through to the loopback interface, 127.0.0.1.
- » **The last rule is for everything else.** All IP addresses will match the destination IP address 0.0.0.0 with subnet mask 0.0.0.0 and will be sent to the default gateway router at 192.168.1.1 via the computer's network interface at 192.168.1.100.



TIP

One major difference between the Windows version of `route` and the Unix/Linux version is the order in which they list the routing table. The Windows `route` command lists the table starting with the most general entry and works toward the most specific. The Unix/Linux version is the other way around: It starts with the most specific and works toward the more general. The Unix/Linux order makes more sense — the Windows `route` command displays the routing list upside down.

Modifying the routing table

Besides displaying the routing table, the `route` command also lets you modify it by adding, deleting, or changing entries.



WARNING

Don't try this unless you know what you're doing. If you mess up the routing table, your computer may not be able to communicate with anyone.

The syntax for the `route` command for adding, deleting, or changing a route entry is

```
route [-p] command dest [mask subnet] gateway [-if interface]
```

The following list describes each of the `route` command's parameters:

- » `-p`: Makes the entry persistent. If you omit `-p`, the entry will be deleted the next time you reboot. (Use this only with add commands.)
- » *command*: Add, delete, or change.
- » *dest*: The IP address of the destination subnet.
- » *mask subnet*: The subnet mask. If you omit the subnet mask, the default is 255.255.255.255, meaning that the entry will apply only to a single host rather than a subnet. You usually want to include the mask.
- » *gateway*: The IP address of the gateway to which packets will be sent.
- » *if interface*: The IP address of the interface through which packets will be sent. If your computer has only one network interface, you can omit this.

Suppose that your network has a second router that serves as a link to another private subnet, 192.168.2.0 (subnet mask 255.255.255.0). The interface on the local side of this router is at 192.168.1.200. To add a static route entry that sends packets intended for the 192.168.2.0 subnet to this router, use a command like this:

```
C:\>route -p add 192.168.2.0 mask 255.255.255.0 192.168.1.200
```

Now, suppose that you later change the IP address of the router to 192.168.1.222. You can update this route with the following command:

```
C:\>route change 192.168.2.0 mask 255.255.255.0 192.168.1.222
```

Notice that I specify the mask again. If you omit the mask from a `route change` command, the command changes the mask to 255.255.255.255!

Finally, suppose that you realize that setting up a second router on this network wasn't such a good idea after all, so you want to just delete the entry. The following command will do the trick:

```
C:\>route delete 192.168.2.0
```

Using the `tracert` Command

The `tracert` command (`traceroute` in Unix/Linux implementations) is one of the key diagnostic tools for TCP/IP. It displays a list of all the routers that a packet must go through to get from the computer where `tracert` is run to any other computer on the internet. Each one of these routers is called a *hop*, presumably because the original designers of the IP protocol played a lot of hopscotch when they were young. If you can't connect to another computer, you can use `tracert` to find out exactly where the problem is occurring.

`tracert` makes three attempts to contact the router at each hop and displays the response time for each of these attempts. Then, it displays the DNS name of the router (if available) and the router's IP address.

To use `tracert`, type the `tracert` command followed by the host name of the host to which you want to trace the route. For example, here's how you can use `tracert` to find out how many routers are between you and the whitehouse's servers:

```
C:\>tracert whitehouse.gov
```

```
Tracing route to whitehouse.gov [2a04:fa87:fffd::c000:42a8]
```

over a maximum of 30 hops:

```

1      1 ms      1 ms      1 ms  2601:204:380:11f0:c294:35ff:fe2a:689e
2     12 ms     13 ms     10 ms  2001:558:4010:33::1
3     11 ms     10 ms     11 ms  po-254-1221-rur01.pinedale.ca.ccal.comcast.net
    [2001:558:212:6801::1]
4     36 ms     19 ms     16 ms  ae-38-ar01.sacramento.ca.ccal.comcast.net
    [2001:558:210:f4::1]
5     17 ms     16 ms     37 ms  be-36441-cs04.sunnyvale.ca.ibone.comcast.net
    [2001:558:3:25b::1]
6     19 ms     16 ms     18 ms  be-1412-cr12.sunnyvale.ca.ibone.comcast.net
    [2001:558:3:38a::2]
7     19 ms     20 ms     18 ms  be-304-cr12.9greateaks.ca.ibone.comcast.net
    [2001:558:3:16d::2]
8     18 ms     27 ms     15 ms  be-1112-cs01.9greateaks.ca.ibone.comcast.net
    [2001:558:3:421::1]
9     18 ms     17 ms     17 ms  be-2101-pe01.9greateaks.ca.ibone.comcast.net
    [2001:558:3:136::2]
10    15 ms     17 ms     17 ms  2001:559::f16
11    16 ms     16 ms     18 ms  2a04:fa87:fffd::c000:42a8

```

Trace complete.

As you can see, getting from my computer to `whitehouse.gov` required 11 hops.

Note that some of the hops in the previous output list IP6 addresses. You can force `tracert` to show you IP4 addresses by using the `-4` flag, like this:

```
C:\>tracert -4 whitehouse.gov
```

```
Tracing route to whitehouse.gov [192.0.66.168]
```

over a maximum of 30 hops:

```

1      1 ms      1 ms      1 ms  10.0.0.1
2     13 ms     10 ms     14 ms  96.120.86.213
3     10 ms      9 ms     10 ms  po-254-1221-rur01.pinedale.ca.ccal.comcast.net
    [96.110.146.25]
4     15 ms     23 ms     22 ms  ae-38-ar01.sacramento.ca.ccal.comcast.net
    [68.87.221.57]
5     19 ms     16 ms     17 ms  be-36441-cs04.sunnyvale.ca.ibone.comcast.net
    [96.110.41.109]
6     16 ms     17 ms     16 ms  be-1412-cr12.sunnyvale.ca.ibone.comcast.net
    [96.110.46.42]
7     17 ms     17 ms     15 ms  be-303-cr12.9greateaks.ca.ibone.comcast.net
    [96.110.37.178]
8     21 ms     17 ms     16 ms  be-1112-cs01.9greateaks.ca.ibone.comcast.net
    [68.86.166.133]

```

```

 9    18 ms    16 ms    17 ms  be-2101-pe01.9greateoaks.ca.ibone.comcast.net
    [96.110.36.218]
10    19 ms    42 ms    17 ms  50.248.116.14
11    15 ms    16 ms    17 ms  192.0.66.168

Trace complete.

```

The most likely problem that you'll encounter when you use `tracert` is a timeout during one of the hops. Timeouts are indicated by asterisks where you'd expect to see a time. For example, the following `tracert` output shows the fourth hop timing out on all three attempts:

```

C:\>tracert -4 whitehouse.gov

Tracing route to whitehouse.gov [192.0.66.168]
over a maximum of 30 hops:

 1     1 ms     1 ms     1 ms  10.0.0.1
 2    13 ms    10 ms    14 ms  96.120.86.213
 3    10 ms     9 ms    10 ms  po-254-1221-rur01.pinedale.ca.ccal.comcast.net
    [96.110.146.25]
 4     *        *        * Request timed out.

```

Sometimes, timeouts are caused by temporary problems, so you should try the `tracert` again to see if the problem persists. If you keep getting timeouts at the same router, the router could be having a genuine problem.

UNDERSTANDING HOW TRACERT WORKS

Understanding how `tracert` works can provide some insight that may help you to interpret the results it provides. Plus, you can use this knowledge to impress your friends, who probably don't know how it works.

The key to `tracert` is a field that's a standard part of all IP packets called TTL, which stands for *Time to Live*. In most other circumstances, a value called TTL would be a time value — not in IP packets, however. In an IP packet, the TTL value indicates how many routers a packet can travel through on its way to its destination. Every time a router forwards an IP packet, it subtracts one from the packet's TTL value. When the TTL value reaches zero, the router refuses to forward the packet.

The `tracert` command sends a series of special messages called ICMP Echo Requests to the destination computer. The first time it sends this message, it sets the TTL value of the packet to 1. When the packet arrives at the first router along the path to the destination, that router subtracts one from the TTL value, sees that the TTL value has become 0, so it sends a Time Exceeded message back to the original host. When the `tracert` command receives this Time Exceeded message, it extracts the IP address of the router from it, calculates the time it took for the message to return, and displays the first hop.

Then the `tracert` command sends another Echo Request message: this time, with the TTL value set to 2. This message goes through the first router to the second router, which sees that the TTL value has been decremented to 0 and then sends back a Time Exceeded message. When `tracert` receives the Time Exceeded message from the second router, it displays the line for the second hop. This process continues, each time with a greater TTL value, until the Echo Request finally reaches the destination.

Pretty clever, eh?

(Note that the Unix/Linux `traceroute` command uses a slightly different set of TCP/IP messages and responses to accomplish the same result.)

3

Planning a Network

Contents at a Glance

CHAPTER 1:	Local Area Networks	233
CHAPTER 2:	Wide Area Networks	249
CHAPTER 3:	Server Architecture	261
CHAPTER 4:	Virtualization Architecture	271
CHAPTER 5:	Storage Architecture	283
CHAPTER 6:	Backup Architecture	295
CHAPTER 7:	Hyperconverged Infrastructure	313